How the standard works

Use this standard to choose valuable work, deliver a small verified improvement, and learn the engineering well enough to reproduce and defend it yourself. The maintained product is this reusable workflow and learning system.

Outcome	Evidence that counts
Business value	Observed task volume, active labor, review, rework and support. Distinguish released capacity from cash savings.
Verified quality	Executed tests, negative cases, source revisions, measured limits and independent review of consequential claims.
Human mastery	Unaided coding or debugging, an architecture defense and a new transfer case. AI-assisted work is recorded separately.

What the repository contains

OpenCode helpers, engineering rules, templates, eight learning modules and dependency-free synthetic reference code. Its current maturity is L1: local reference checks. Version 2 does not mean enterprise production readiness.

What you supply in a company

A bounded approved task, source owners, real identity and authorization, domain validators, operating ownership and a usable employee workflow. The repository provides no production executor, company approvals, live model or employee application.

Start here
Read pages 1-3 for the operating model, 4-8 for design and value, 9-10 for daily and offline use, and 11-12 for interview preparation. The guide explains canonical rules; its source list on every page tells you where the full rule lives.

Sources AGENTS.md
README.md
docs/GOVERNANCE.md

Run one bounded change

Each iteration produces a reviewable result and a learning artifact. Specify acceptance before writing code; preserve working architecture and model settings.

Step	Question to answer	Artifact
Inspect	What exists, works or fails? What is authorized?	Git state, code/test evidence, gap matrix
Specify	Who benefits, and what would prove success?	Charter, baseline, scope and acceptance
Design	What is the simplest reversible approach?	Alternative, failure model, ADR if consequential
Implement	What is the smallest coherent change?	Bounded diff; preserved settings
Verify	Did the actual checks pass, including failures?	Executed commands, outcomes, unresolved gaps
Measure	What changed in quality, latency and net effort?	Before/after with method and uncertainty
Master	Can you reproduce and defend the mechanism?	Unaided lab, defense, dated readiness record

A synthetic example

A search result incorrectly boosts JOB-1 when the query asks for JOB-10. Reproduce the failure, define token-equality acceptance, make the small fix and rerun regressions. Then independently explain tokenization, ranking cost and why the toy role filter is not authentication.

Finish honestly
Report verified behavior, exact tests, unverified integrations, measurements and one next task. A merged PR is software evidence; it is not proof that the human mastered the code.

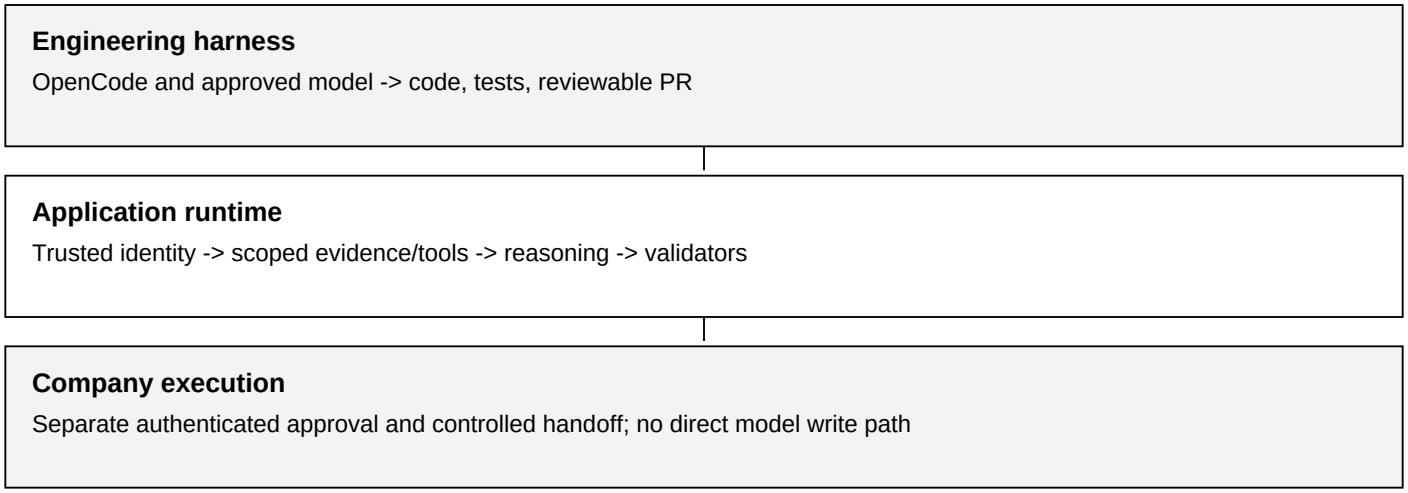
Sources AGENTS.md
docs/DEFINITION_OF_DONE.md
templates/CHARTER.md

Keep authority outside the model

Separate the engineering assistant, the application runtime and company-controlled execution. The model can propose; trusted code and accountable people enforce the boundaries.

Model workflow and agent

A language model generates outputs from context. A fixed workflow runs known steps. An agent uses model decisions and tool feedback to choose its next step within a bounded goal. Retrieval supplies evidence; it does not make the model authoritative. Keep deterministic work deterministic, and add dynamic decisions only when they help.



The local teaching state machine

ChangeRun models created -> evidence_ready -> drafted -> validated -> reviewed -> exported. It rehashes the draft at gates. Reviewed is an unverified caller assertion; exported is a returned local copy. Fields and events remain mutable. There is no durable store, real approval, retry deduplication or audit-integrity guarantee.

The target production contract

Task requests carry identity references, scope, budgets, deadlines and idempotency keys. Evidence carries revision, hash, permission scope and timestamps. Tool calls and traces bind approved definitions and policy decisions. Approval binds exact artifact, validator, reviewer, scope and expiry. Cancellation competes atomically with handoff.

Interview defense
Why does independent=True fail to prove reviewer independence? Why can a timeout mean an unknown remote outcome? Explain the transaction and trusted identity boundary before discussing more agents.

Sources docs/ARCHITECTURE.md
docs/RUNTIME_CONTRACTS.md
golden/change.py

Choose the smallest useful pattern

Start with deterministic lookup or retrieval. Add autonomy only for a demonstrated need, and measure the coordination overhead against the simpler baseline.

Pattern	Use when	Main cost or failure
Simple retrieval	IDs or evidence lookup satisfy the task	Lexical misses; hits do not prove an answer
Single agent	Next approved read depends on evidence	Loops, stochastic choices, serial latency
Sequential chain	Stages are known in advance	Serial delay and propagated stage errors
Routing	Stable task classes need distinct tools	Misclassification; routing grants no access
Parallel fan-out/in	Read tasks are independent	Extra calls, partial results, correlated errors
Orchestrator-workers	Decomposition emerges during work	Delegation cost and inconsistent context
Evaluator-optimizer	Output can improve against a trusted rubric	Judge bias, repeated cost, overfitting
Durable workflow	Work spans crashes, schedules or human waits	State storage, replay and migrations

Bound every design

Declare total step, tool, time and cost limits. Stop on policy denial, cancellation, deadline, exhausted budget or no progress. Examples in the pattern guide are starting limits, not universal SLOs. Durable execution can wrap any pattern without adding a model.

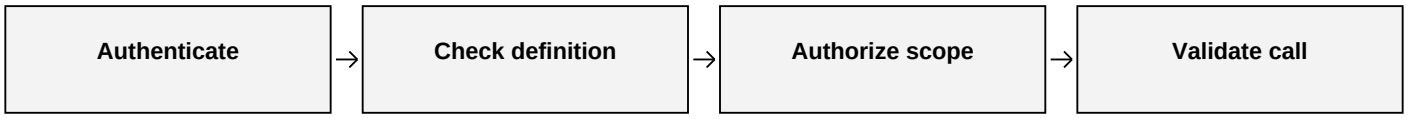
Decision to defend
State why your chosen pattern beats search, a script or a fixed workflow on the same cases. More agents are justified by observed benefit after coordination cost.

Sources docs/AGENT_PATTERNS.md
docs/PERFORMANCE_PLAYBOOK.md

Build context and govern tools

Retrieved material is evidence, even when it contains text pretending to be a system instruction. It cannot edit policy, tool definitions, permissions or workflow truth.

Context plane	Authority and lifetime
Permanent rules	Reviewed project/company policy; versioned and reconciled
On-demand skills	Approved pinned publication; selected for the task
Retrieved knowledge	Authorized revision-pinned data; request-scoped and untrusted as instructions
Tools	Reviewed server and definition registry; minimal exposed operations
Permissions	Trusted identity/policy service; checked per call, resume and output
Workflow state	Server-owned versioned checkpoint; model proposes transitions



A definition hash needs a trusted baseline

Review descriptions and schemas before enrollment. Deny unknown servers/tools and definition drift. Check classification and approved model destination before disclosure. Keep secrets outside model context; enforce permissions again at the adapter and source. Tool annotations and retrieved commands do not authorize effects. MCP is a protocol for connecting tools and resources; it does not itself approve a server or grant business permissions.

What the offline tests establish
The policy example denies poisoned definitions, disallowed scope, classification, budget and canceled calls. It does not authenticate users, connect to MCP, execute tools or prove that a live model resists prompt injection.

Sources docs/CONTEXT_ENGINEERING.md
docs/TOOL_TRUST.md
golden/tools.py

Measure the right kind of success

Separate contract correctness, retrieval, answer grounding, workflow completion and business impact. Success at one layer does not establish the next.

Layer	Useful check	Limit
Contracts	Types, hashes, state order, scope and cancellation	Only the tested boundaries
Retrieval	Recall, precision, MRR and leakage by slice	Correct IDs do not prove correct claims
Answer	Claim correctness, citation support and revision	Needs labeled answers and calibrated graders
Workflow	Safe completion, latency, recovery, tool usage/cost	Needs the whole employee journey
Business	Net active effort, quality, adoption and support	Needs an observed baseline/comparison

A metric counterexample

Expected relevant IDs are A, B and C. Top three returned IDs are X, A and B. Recall@3 is 2/3, precision@3 is 2/3, reciprocal rank is 1/2. MRR averages reciprocal rank across answerable cases. Evaluate correct abstention separately for questions with no expected evidence.

Know the provided evaluator

The seven original cases and 30 public domain-seed cases test tiny synthetic lexical retrieval. The legacy hit_at_k field combines a hit with correct abstention; it is not recall. The runner reports hashes, k, repeats and query-only p50/p95. No answer model runs; answer correctness, grounding and task success are unmeasured.

Evidence quality
Public fixtures are not protected holdouts. Repeated deterministic timing is not repeated model trials. Fresh holdouts require a separate curator; human/AI grading needs calibration. Report actual failures and provenance.

Sources docs/QUALITY_AND_EVALUATIONS.md
golden/eval.py

Select work that repays its cost

Compare a small set of opportunities, including doing nothing and a deterministic improvement. Use a common horizon and label inputs measured, estimated or unknown.

Synthetic input	Value
Eligible tasks per month	40
Manual active minutes per task	20
Assisted + incremental review + rework	6 + 4 + 2
Monthly support minutes	120
One-time build minutes	480

How the example works

40 x (20 - 6 - 4 - 2) = 320 minutes/month before support. Subtract 120 support minutes: 200 net minutes/month. Build effort 480 / 200 = 2.4 months to break even, assuming stable demand and no other costs. First-month net after build is -280 minutes. These invented inputs illustrate arithmetic, not achieved ROI.

Choose under uncertainty

Gate on authorization, safety, data and operating ownership. Compare conservative net benefit, time to evidence and maintenance burden. If unknown adoption changes the ranking, measure it first. Count review/rework once. Released capacity is not automatically cash saved. Yield or defect value requires independent attribution.

Interview defense
What assumption would reverse your recommendation? Why does an unknown cost differ from zero? What evidence would justify an agent over an approved export and a simple script?

Sources docs/IMPACT_AND_EXECUTIVE_RECOGNITION.md
templates/EXPERIMENT.md

Adopt with limited IT support

Use what the organization can support. An approved versioned snapshot can test usefulness before a live connector. A shared service still needs real identity, permissions and an operator.

Constraint	Smallest credible response
No approved API	Approved scoped export; record revision, age and missing data
Little support capacity	One supported path, manual recovery, named owner and handover
Existing code/config	Audit, dry-run retrofit, reconcile collisions; preserve settings
Ordinary employees	Complete accessible UI journey, understandable evidence and recovery
Unclear value	One measured task with review/rework before wider rollout

Semiconductor example

On a synthetic source revision, investigate how sample_count flows through config, parser and validator. Cite exact revision and distinguish static evidence from unknown runtime dispatch. Compare two revisions and an override. This is an example for a separately authorized consuming application, not an assignment to build one in the standard.

Transfer beyond manufacturing

For maintenance-manual search, preserve evidence revisions, authorization, uncertainty and evaluation. Change the task baseline, document owner and consequences of a wrong answer. Reuse contracts after validating the second setting; unobserved reuse is not realized business value.

Promotion requires evidence
Advance stages on observed quality, safe completion, sustainable support and net benefit. A passing local fixture never authorizes equipment, recipe, MES or production database writes.

Sources docs/PRODUCT_DELIVERY.md
docs/BROWNFIELD_ADOPTION.md
docs/playbooks/MANUFACTURING_SYSTEM_INTELLIGENCE.md

Find the right file and command

Keep the immutable reference snapshot separate from the application and from your learning records. Use canonical sources when a summary is not enough.

Need	Canonical starting point
Work loop and authority	AGENTS.md
Release evidence and review	docs/DEFINITION_OF_DONE.md and docs/GOVERNANCE.md
Architecture and runtime messages	docs/ARCHITECTURE.md and docs/RUNTIME_CONTRACTS.md
Pattern, context and tools	docs/AGENT_PATTERNS.md, CONTEXT_ENGINEERING.md, TOOL_TRUST.md
Measurement and value	docs/QUALITY_AND_EVALUATIONS.md and IMPACT_AND_EXECUTIVE_RECOGNITION.md
Learning and current ability	academy/README.md and templates/READINESS_SNAPSHOT.md

OpenCode command	Purpose
/golden-audit	Inspect authorized existing behavior and evidence
/golden-next	Choose one scoped, valuable next standard improvement
/golden-build	Implement and verify a bounded change
/golden-eval	Design/run an evidence-based evaluation
/golden-security	Review trust, access, data exposure and effects
/golden-master	Teach, assign unaided work and assess evidence honestly
/golden-weekly	Review value, quality, ability and one next task

Helpers are optional
OpenCode needs approved tooling/model access. You can read the guide, run synthetic Python checks and do the exercises without it. Commands and skills never grant company permissions.

Sources AGENTS.md
.opencode/commands/
docs/GOVERNANCE.md

Start on the Windows work PC

Use the approved one-way inbound-transfer process. This package does not need information returned from the work PC.

Step	Action
Before copy	Confirm approved Python 3.11+ is already available. PDF reading alone needs only a reader.
Check ZIP	Compare Get-FileHash output against the separately trusted expected SHA-256.
Extract	Use a new folder and approved Windows extraction tool; open PowerShell there.
Preflight	Run the single command below. Stop on failure; retain logs locally.
Preserve	Keep source snapshot unchanged; work in workspace/ and store results in out/ or results/.

```
py -3 --version
py -3 scripts/offline_preflight.py --transfer
```

What the preflight does

Checks the file manifest, structure/version/skill hashes, unit tests, both fixture sets and synthetic demos. It downloads nothing and invokes no model. A publisher-only packaging test may skip if Git is absent. Python itself, OpenCode, model weights, credentials and company integrations are not bundled.

Understand the assurance

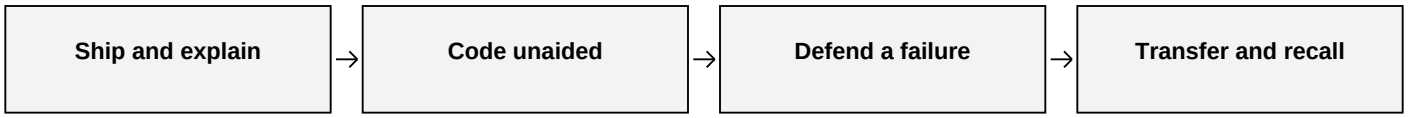
The ZIP names an exact committed snapshot. Checksums detect changes against a trusted expected value; they are not independent approval or publisher authentication. Windows execution is unverified until run on that PC. macOS and Linux CI results do not substitute for it.

If a prerequisite is blocked
Use approved internal provisioning or keep to reading and paper exercises. Preserve device controls and existing model settings. Detailed commands and brownfield steps are in OFFLINE_START.md.

Sources OFFLINE_START.md
scripts/offline_preflight.py
scripts/verify_transfer.py

Build interview readiness through work

Target a role family, level and format. Keep a dated evidence index so you can see what is independent, assisted, stale or unassessed at any time.



Rubric level	Observed evidence
0 Not assessed	No evidence yet
1 Conceptual	Explains with prompts/notes; cannot implement independently
2 Assisted	Builds with AI; cannot reproduce independently yet
3 Independent	Unaided implementation/debugging plus adversarial tests and defense
4 Production and leadership	Owns real release/support and measured impact; defends tradeoffs

Build breadth with specific evidence

Applied agent roles emphasize bounded workflows and answer quality; platform/backend roles add concurrency and recovery; retrieval/evaluation roles add ground truth and uncertainty; security/reliability roles add abuse and fault diagnosis; technical leadership adds alternatives, ownership and adoption. Actual role requirements may add ML research, mathematics or accelerator infrastructure beyond this academy.

A useful practice cadence

After a meaningful change, explain the mechanism, complete one unaided lab and defend a new failure case. Practice recall on days 2, 7 and 30. Suggested upkeep is a weekly mixed drill and monthly unseen coding/design assessment. These are local practice defaults, not a hiring guarantee.

Prepare an evidence packet
A 90-second project explanation, five-minute failure/tradeoff defense, executable unaided artifact and source-approved impact story. Keep company material inside approved systems; external career evidence needs appropriate approval.

Sources academy/README.md
academy/INTERVIEW_BANK.md
templates/MASTERY_RECORD.md
templates/READINESS_SNAPSHOT.md

Complete your first unaided session

The first assessment observes validation, missing-evidence reasoning, testing and value judgment. It contains no generated solution and does not assign you a score.

Time box	Work to produce
5 minutes	Read academy/assessment/README.md; predict the example before coding.
25 minutes	Write assess(case) in workspace/my_solution.py without AI-generated code.
10 minutes	Run visible checks; diagnose failures; retain actual test output.
10 minutes	Defend None vs zero, bool vs int, overflow, costs and operating ownership.
5 minutes	Record evidence/hints/gaps and schedule recall; assessor adds a fresh case.

```
py -3 academy/assessment/check_submission.py workspace/my_solution.py
```

What the function must decide

Validate the exact synthetic input schema. Reject invalid known values before handling unknowns. Return insufficient_evidence when a field is unknown. Otherwise compute monthly net capacity and break-even, preserving zero/negative benefit. The full contract and public examples are in the assessment README.

Checks are not the whole assessment

Visible tests let you self-check the contract. Independent mastery also requires observed unaided work, a new failure/transfer case and architecture defense. Record actual results in a dated copy of the readiness template. Next deeper lab: the SQLite retry, concurrency and cancellation exercise.

Explain this standard without notes
What do you choose before using an agent? Where does authority live? What proves an answer is grounded? Which cost could erase the benefit? What can you implement independently today? Use gaps to select the next standard improvement.

Sources academy/assessment/README.md
academy/V2_INTEGRITY_EXERCISE.md
academy/README.md